



Atividade 6 — Ingestão ETL

Python + PySpark + AWS

Ambiente Docker local com processamento Spark e carga em Amazon S3

Grupo 6
Engenharia de Dados + Big Data
Universidade de São Paulo — Escola Politécnica (USP – POLI)



Demonstração acadêmica • execução local + integração Cloud

Objetivo da atividade

Construir um pipeline ETL reproduzível

- Executar Apache Spark em containers Docker.
- Ler diferentes fontes e formatos de dados.
- Aplicar limpeza, padronização e transformação com PySpark.
- Gerar Parquet e particionar os dados por Ano e Trimestre.
- Disponibilizar os dados processados no Amazon S3.
- Separar código, configuração e credenciais.

EXTRACT

CSV / TSV
→ DataFrames Spark

TRANSFORM

Limpeza
JOIN
Indicadores

LOAD

Parquet
Particionado

CLOUD

Amazon S3
s3a://

Arquitetura pensada para estudo e baixo consumo de recursos AWS.

Conjunto fornecido para o laboratório

Dados trimestrais

2021_tri_01.csv
2021_tri_02.csv
2021_tri_03.csv
2021_tri_04.csv
2022_tri_01.csv
2022_tri_03.csv
2022_tri_04.csv

Separador: ;

Enquadramento

EnquadramentoInicia_v2.tsv

Campos principais:

- Segmento
- CNPJ
- Nome

Formato TSV / TAB

Glassdoor

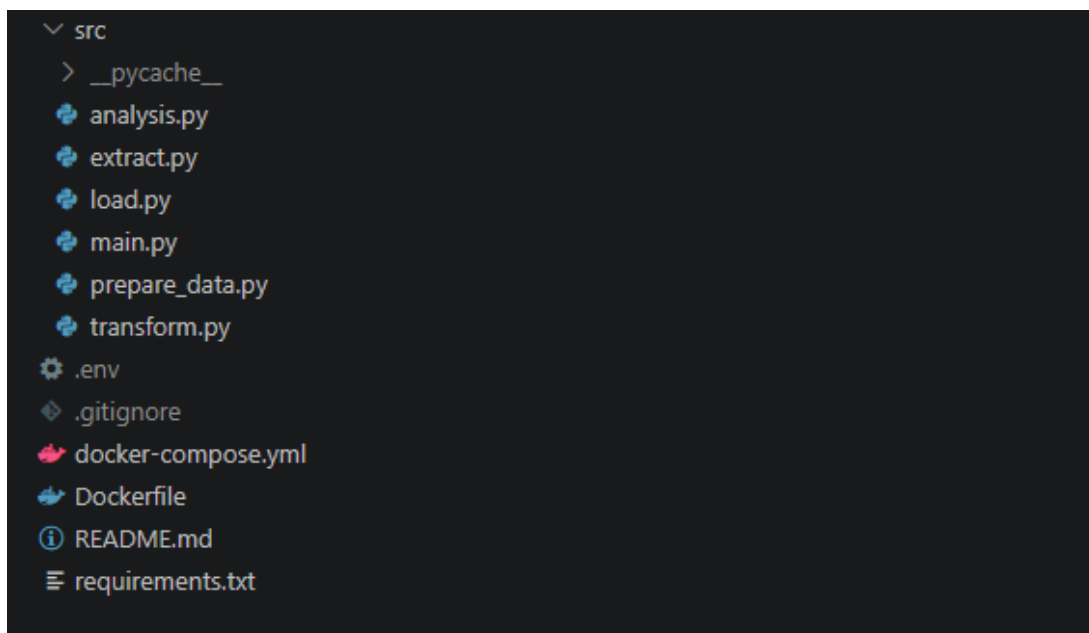
glassdoor_consolidado_join_match_v2.csv
glassdoor_consolidado_join_match_les_s_v2.csv

Informações de empresa, avaliações e correspondência.

Observação: o conjunto disponibilizado não contém 2022_tri_02.csv.

Ambiente local no Docker

Estrutura do projeto no VS Code



Docker Compose

Spark Master + Spark Worker

Código ETL

prepare_data.py • extract.py • transform.py • load.py • main.py • analysis.py

Configuração

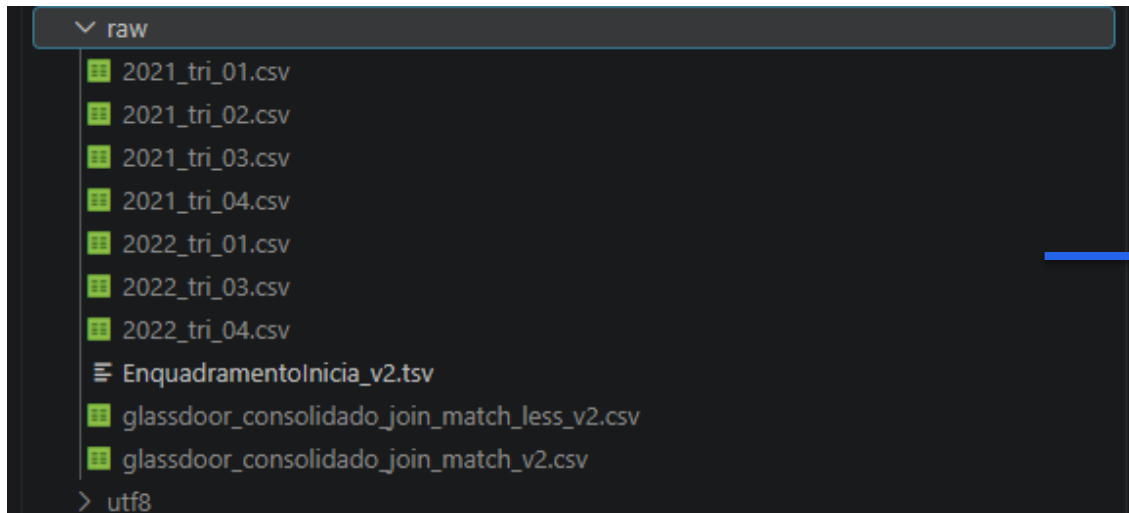
.env • .gitignore • Dockerfile • requirements.txt • README.md

O processamento fica isolado em containers e o projeto mantém código e configuração separados.

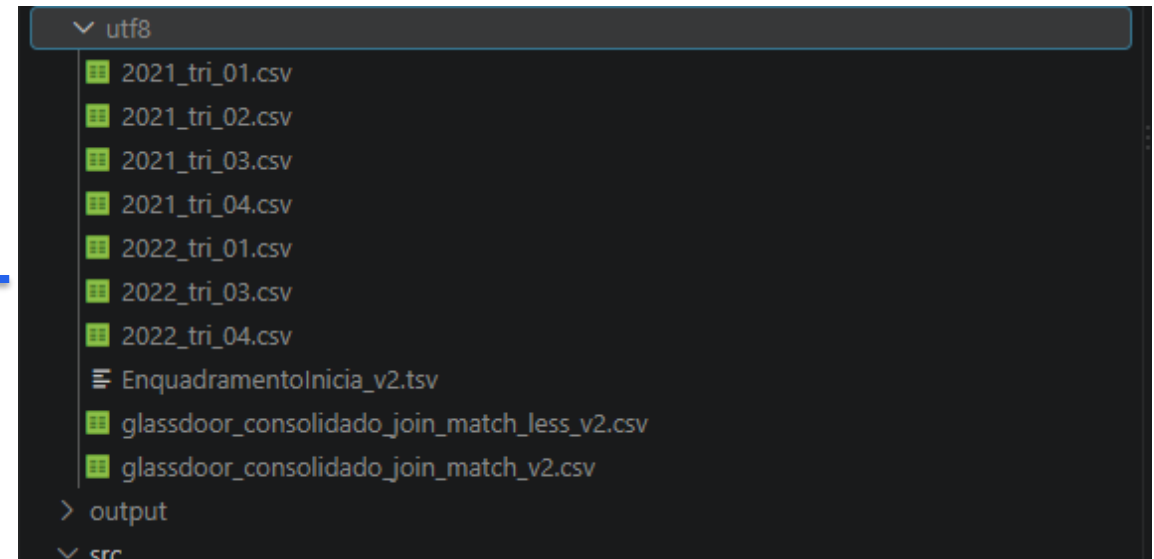
Preparação e organização dos dados

Arquivos originais e arquivos normalizados

data/raw/



data/utf8/



A etapa de preparação normaliza os arquivos para facilitar a leitura pelo Spark, preservando a separação entre origem e dados preparados.

Arquitetura do pipeline ETL

Fluxo lógico implementado



Princípio: processamento local reproduzível + armazenamento Cloud desacoplado do código.

LOAD: local + Amazon S3

O mesmo código suporta dois destinos

```
LOAD_MODE = os.getenv("LOAD_MODE", "LOCAL")
AWS_BUCKET = os.getenv("AWS_BUCKET", "")

def build_output_path(dataset_name):
    if LOAD_MODE == "LOCAL":
        return f"{BASE_OUTPUT}/{dataset_name}"

    return (
        f"s3a://{AWS_BUCKET}/"
        f"processed/{dataset_name}"
    )

def save_parquet(df, dataset_name, partition_columns=None):
    output_path = build_output_path(dataset_name)

    writer = df.write.mode("overwrite")

    if partition_columns:
        writer = writer.partitionBy(*partition_columns)

    writer.parquet(output_path)
```

LOCAL

data/processed/reclamacoes

S3

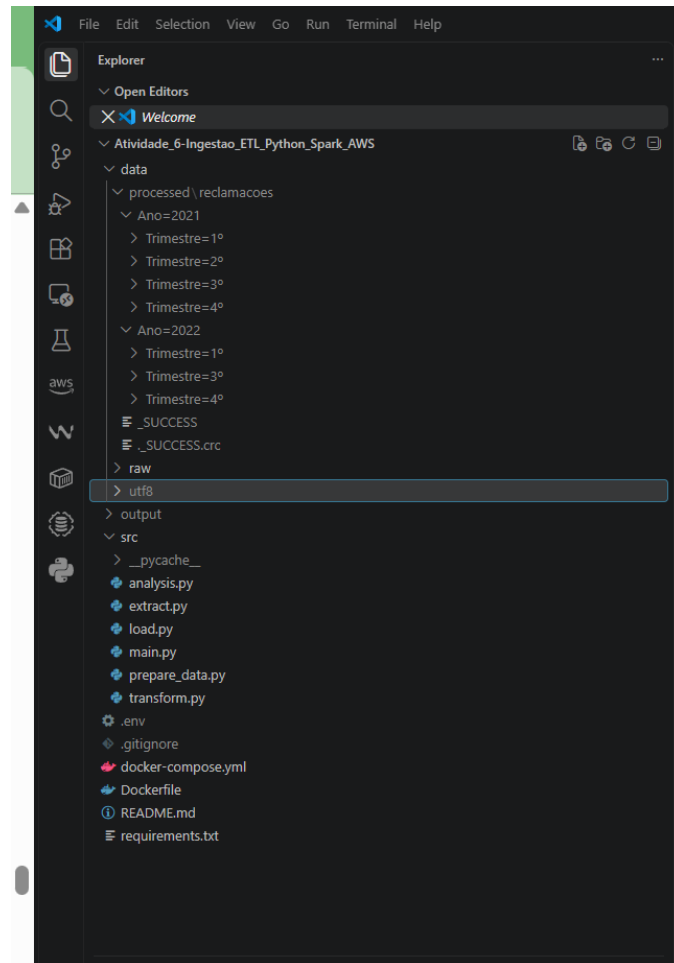
s3a://etl-pyspark-python/processed/reclamacoes

Segurança

Credenciais por variáveis de ambiente; não ficam no código.

Resultado do processamento local

Parquet particionado gerado pelo pipeline



Ano=2021

Trimestre=1º, 2º, 3º e 4º

Ano=2022

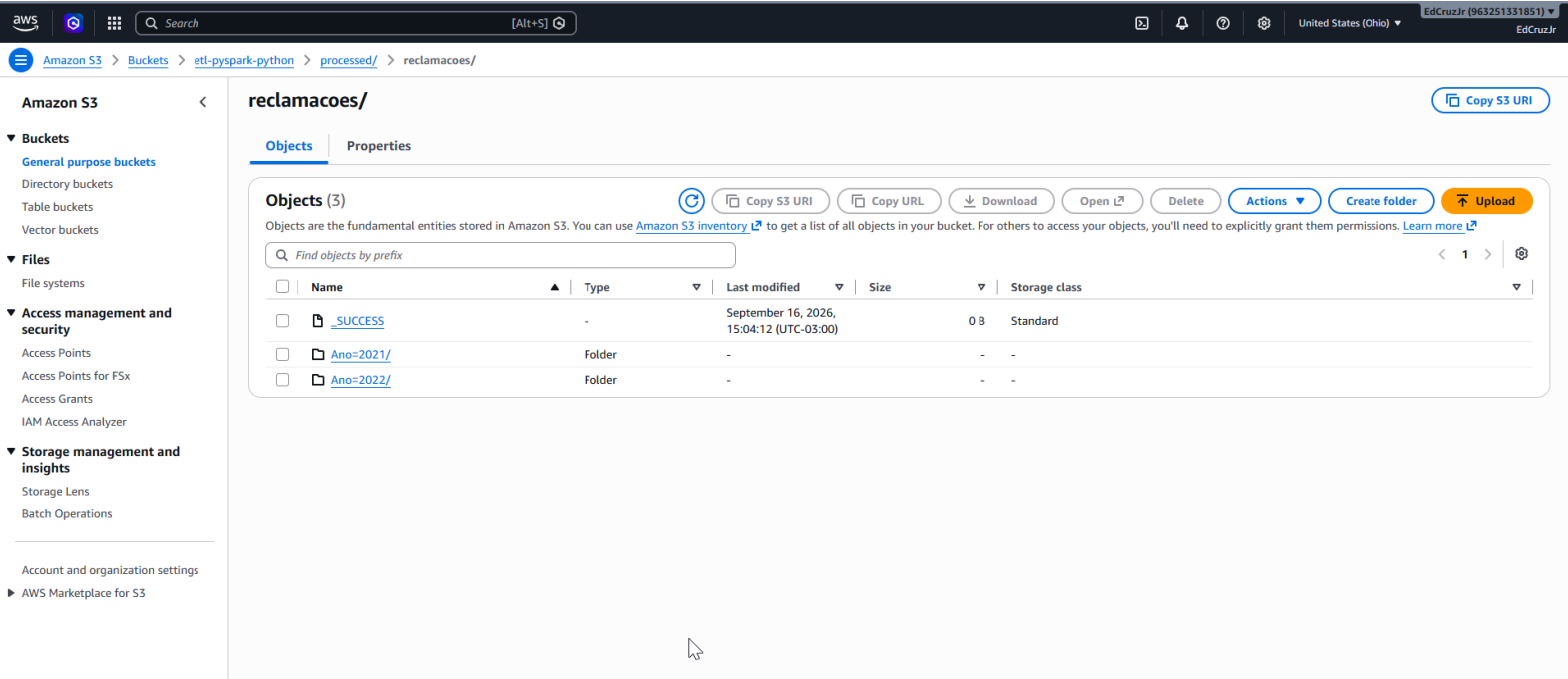
Trimestre=1º, 3º e 4º

Formato

Arquivos part-*.parquet + marcador
_SUCCESS

Carga realizada no Amazon S3

Bucket: etl-pyspark-python • prefixo: processed/reclamacoes/



AWS S3

Resultado disponível no bucket Cloud.

Partições

Ano=2021 e Ano=2022.

Parquet

Objetos part-*.snappy.parquet.

Print do Console AWS utilizado como evidência da carga.

Consulta do conteúdo do S3 após a execução

```
What's next:
  Try Docker Debug for seamless, persistent debugging tools in any container or image → docker debug spark-master
  Learn more at https://docs.docker.com/go/debug-cli/
● PS C:\Users\edvaldo.cruz\Documents\Engenharia_de_Dados\EDB-022-2026-3-Ingestao_Pipeline_Dados\Atividade_6-Ingestao_ETL_Python_Spark_AWS> aws s3 ls s3://etl-pyspark-python/processed/ --recursive
2026-09-16 15:04:09      14608 processed/reclamacoes/Ano=2021/Trimestre=1º/part-00006-df0049fa-b310-466b-8a62-abb4f65140ed.c000.snappy.parquet
2026-09-16 15:04:04      15287 processed/reclamacoes/Ano=2021/Trimestre=2º/part-00005-df0049fa-b310-466b-8a62-abb4f65140ed.c000.snappy.parquet
2026-09-16 15:03:59      15480 processed/reclamacoes/Ano=2021/Trimestre=3º/part-00004-df0049fa-b310-466b-8a62-abb4f65140ed.c000.snappy.parquet
2026-09-16 15:03:54      17747 processed/reclamacoes/Ano=2021/Trimestre=4º/part-00003-df0049fa-b310-466b-8a62-abb4f65140ed.c000.snappy.parquet
2026-09-16 15:03:50      17658 processed/reclamacoes/Ano=2022/Trimestre=1º/part-00002-df0049fa-b310-466b-8a62-abb4f65140ed.c000.snappy.parquet
2026-09-16 15:03:40      19656 processed/reclamacoes/Ano=2022/Trimestre=3º/part-00000-df0049fa-b310-466b-8a62-abb4f65140ed.c000.snappy.parquet
2026-09-16 15:03:45      19023 processed/reclamacoes/Ano=2022/Trimestre=4º/part-00001-df0049fa-b310-466b-8a62-abb4f65140ed.c000.snappy.parquet
2026-09-16 15:04:12           0 processed/reclamacoes/_SUCCESS
○ PS C:\Users\edvaldo.cruz\Documents\Engenharia_de_Dados\EDB-022-2026-3-Ingestao_Pipeline_Dados\Atividade_6-Ingestao_ETL_Python_Spark_AWS> []
```

```
aws s3 ls s3://etl-pyspark-python/processed/ --recursive
```

Evidência observada

Objetos Parquet listados no S3, organizados por Ano e Trimestre.

Responsabilidade dos componentes

RESUMO TÉCNICO

prepare_data.py

Normalização dos arquivos antes do processamento.

extract.py

Leitura das fontes e criação dos DataFrames Spark.

transform.py

Limpeza, tipos, indicadores e preparação para integração.

load.py

Persistência em Parquet local ou S3 via s3a://.

main.py

Orquestração das etapas do pipeline.

analysis.py

Consultas e análises sobre os dados processados.

Infraestrutura

Docker Compose
Spark Master
Spark Worker
Volumes locais

Cloud

Amazon S3
IAM
S3A
Variáveis de ambiente

Resultados e conclusão

O que foi demonstrado

- Ambiente Spark reproduzível em Docker.
- Pipeline ETL desenvolvido em Python + PySpark.
- Leitura de múltiplos arquivos e delimitadores.
- Transformação e preparação dos dados.
- Geração de Parquet com particionamento.
- Carga local e integração com Amazon S3.
- Credenciais desacopladas do código.



Próximas evoluções

Airflow • Great Expectations • Lambda • SQS/DLQ •
CloudWatch • Glue • Athena

A solução cria uma base para evoluir de um laboratório local para uma arquitetura de dados Cloud mais completa.